深度學習
Deep Learning

⭐ DL Training Tips

Instructor: 林英嘉 (Ying-Jia Lin)
2025/10/15

Course GitHub

Slido # DL1015

Outline

- Model training tips [70 min]
  - EarlyStopping
  - Regularization and AdamW
  - Batch Normalization
  - Dropout
- Code demo [10 min]
- Term Project [50 min]
- Quiz [20 min]

[Recap] 如何訓練模型：資料集

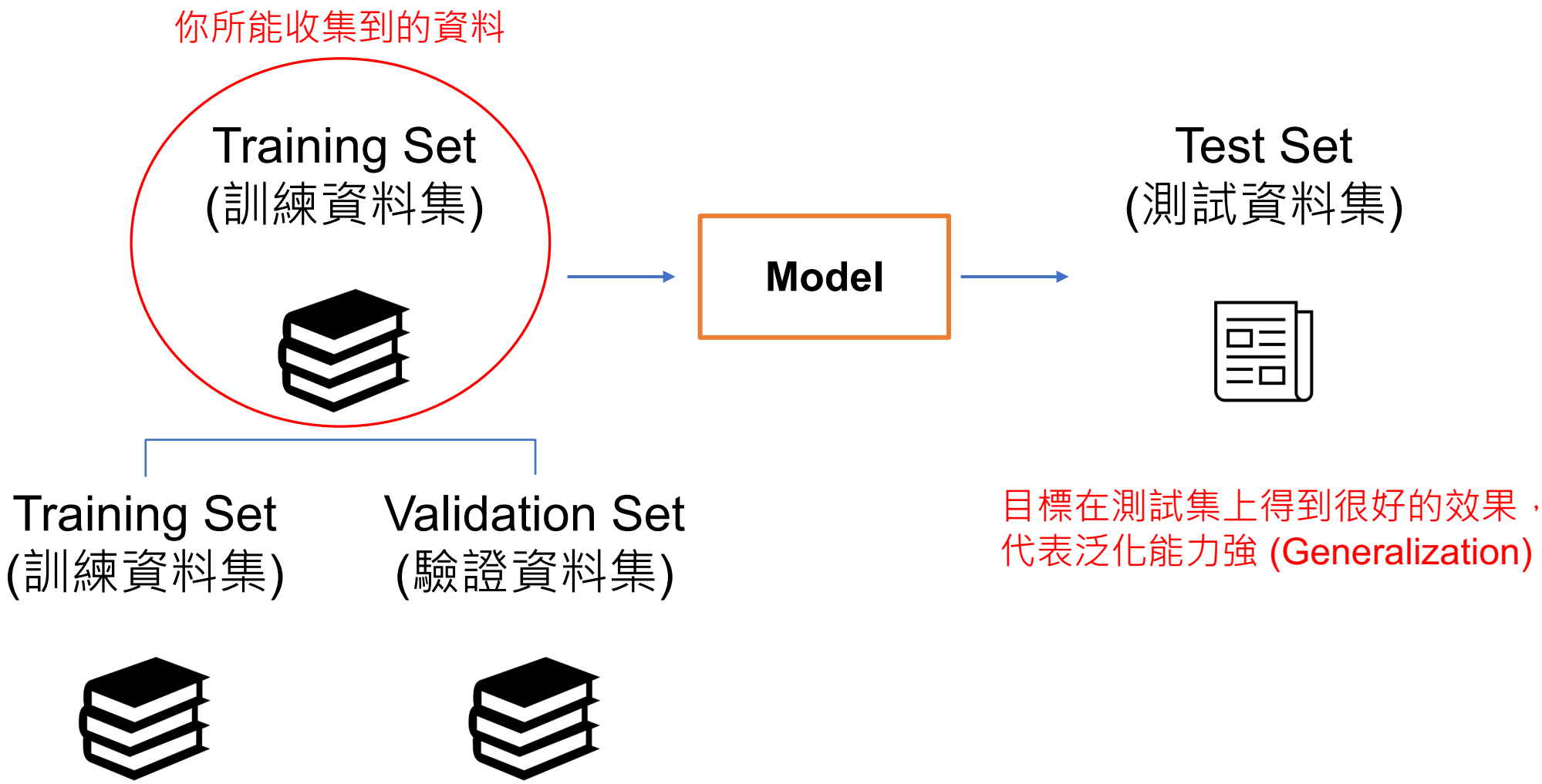
Training Set
(訓練資料集)

Test Set
(測試資料集)

Training Set
(訓練資料集)

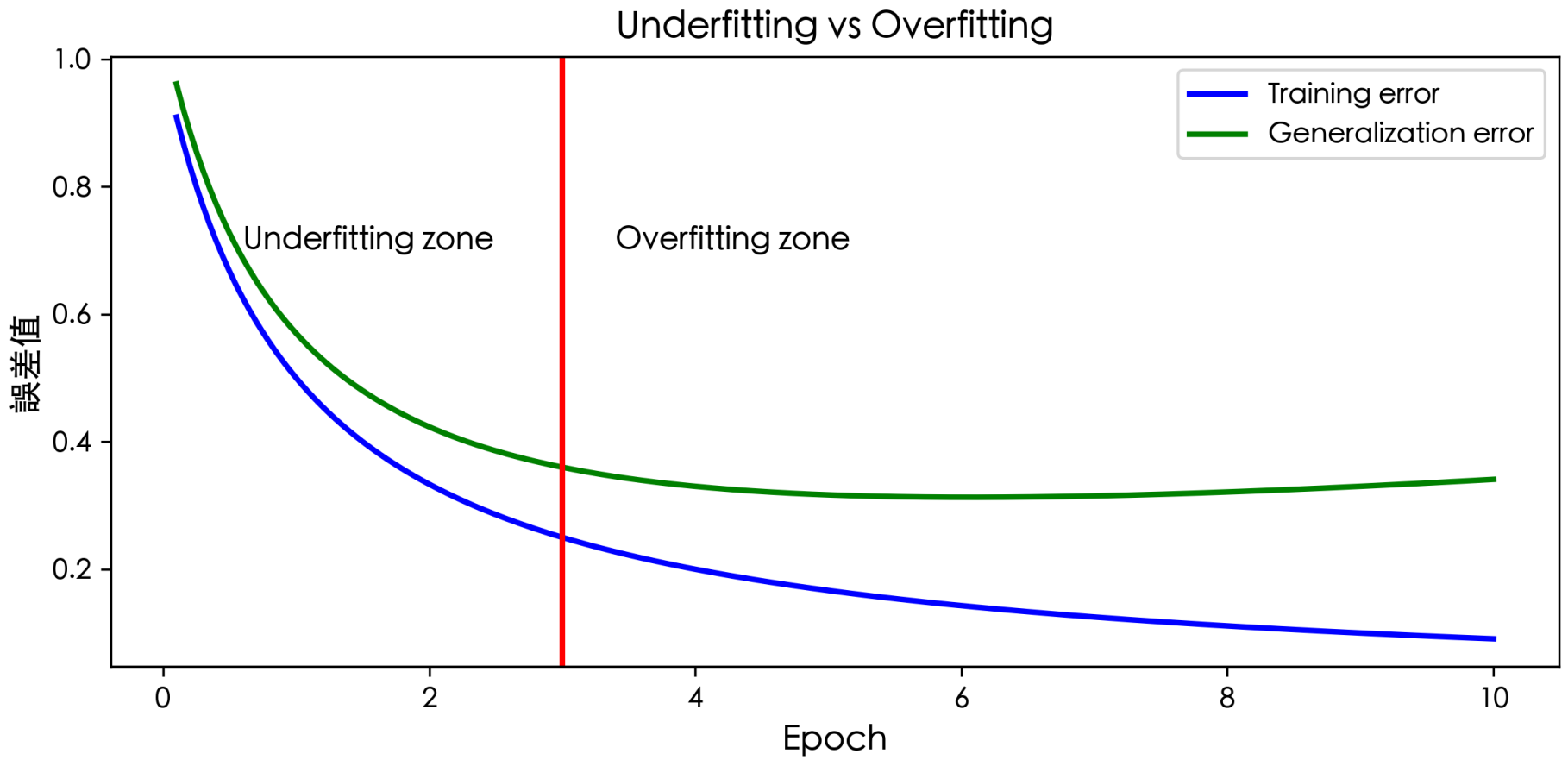
Validation Set
(驗證資料集)

訓練模型的目標


你所能收集到的資料
Training Set
(訓練資料集)
Model
Test Set
(測試資料集)
Training Set
(訓練資料集)
Validation Set
(驗證資料集)
目標在測試集上得到很好的效果，
代表泛化能力強 (Generalization)

訓練機器學習模型時可能發生的情況

code/w7_plot_overfitting.py

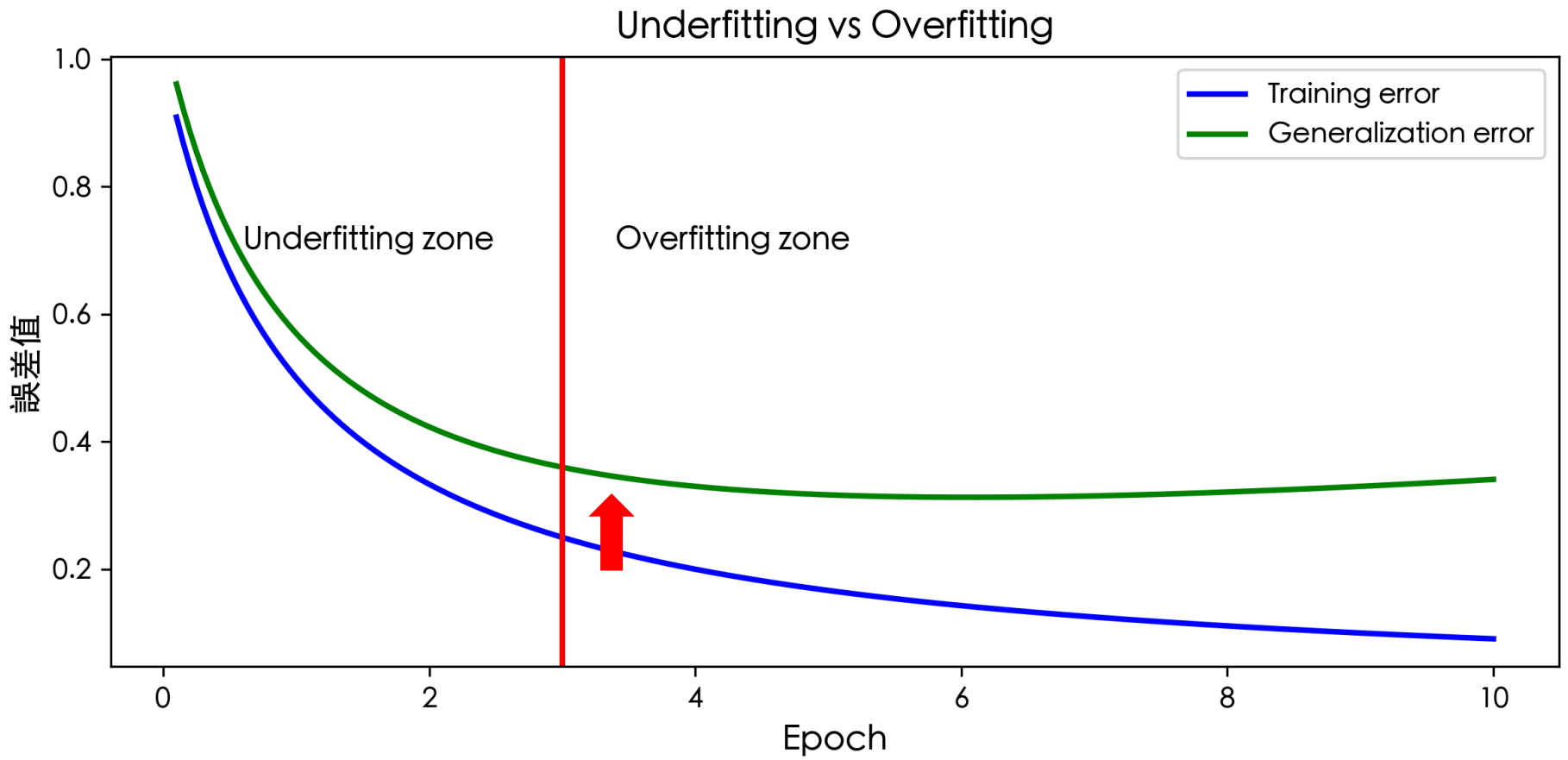


[定義] Overfitting 與 Underfitting

- Underfitting (欠擬合): 模型無法在 training set 取得很低的誤差 (error; loss)
- Overfitting (過擬合): 模型的 training error 與 test error 差異很大
  - test error 通常代表 validation loss
  - 常見模型可以呈現很小的 training error，但 test error 仍大

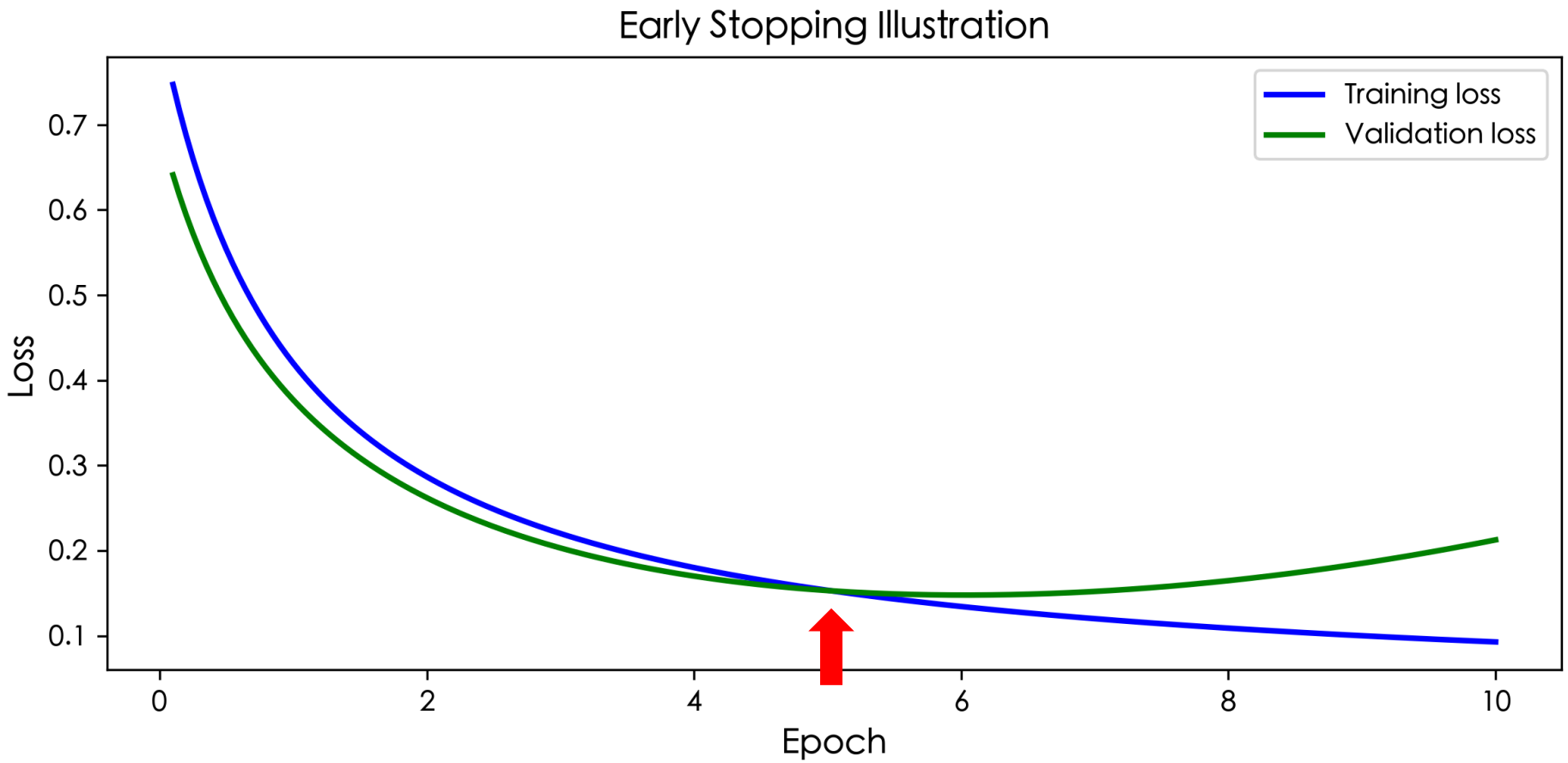
EarlyStopping

code/w7_plot_overfitting.py



EarlyStopping

code/w7_plot_es.py



Regularization

- 定義：regularization (正則化) 是一種技巧來幫助模型減少 training error 與 test error 的差距 (並非降低 training error)

$$\mathcal{L}_{\text{new}} = \mathcal{L} + \boxed{\text{regularization\_term}}$$

Norm (範數)

是具有「長度」概念的函數，其為向量空間內的所有向量的正長度或大小。

假設有一向量 $\mathrm{w} = [w_1, w_2, \ldots, w_n]$

$$\text{L}^1\text{-Norm} = \sum_{i=1}^{n} |w_i| = |w_1| + |w_2| + \cdots + |w_n|$$

$$\text{L}^2\text{-Norm} = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$$

L^2 Regularization

$\mathrm{w} = \{w_1, w_2, \ldots, w_n\}$
λ: 正則項的比例 (0-1之間)

$$\mathcal{L}_{\text{new}} = \mathcal{L} + \lambda \frac{1}{2} \|\mathrm{w}\|_2^2$$

$$w_1^2 + w_2^2 + \cdots + w_n^2$$

代表模型的目標除了要讓 $\mathcal{L}$ 越小越好之外，
還要讓所有的 w 的平方和越小越好

L^2-Norm 說明：https://mathworld.wolfram.com/L2-Norm.html

為什麼要增加正則項？

假設我們的模型：$\hat{y} = x_1 w_1 + x_2 w_2^2 + \cdots + x_9 w_9^9$

如果沒有正則項，目標函數相當於
是為 training set 量身打造

如果 w_9很大怎麼辦？
正則項 (如 L^2) 能限制
weight 數值大小

假設我們想要學出的模型：$y = x_1 w_1 + x_2 w_2^2$

非常通用的模型，
可以在 unseen 資料上表現好

Gradient Descent with L^2 Regularization

$$\mathcal{L}_{\text{new}} = \mathcal{L} + \lambda \frac{1}{2} \|\mathrm{w}\|_2$$

對任意 w_i 進行偏微分：

$$\frac{\partial \mathcal{L}_{\text{new}}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial w_i} + \lambda w_i$$

梯度下降：

$$w_i' = w_i - \eta \frac{\partial \mathcal{L}_{\text{new}}}{\partial w_i}$$

(代入) $$= w_i - \eta\left(\frac{\partial \mathcal{L}}{\partial w_i} + \lambda w_i\right)$$

(移項) $$= \underline{(1 - \eta\lambda)} w_i - \eta\left(\frac{\partial \mathcal{L}}{\partial w_i}\right)$$

每次更新都會倍數遞減
(weight decay)

L^1 Regularization

$\mathrm{w} = \{w_1, w_2, \ldots, w_n\}$
λ: 正則項的比例 (0-1之間)

$$\mathcal{L}_{\text{new}} = \mathcal{L} + \lambda \|\mathrm{w}\|_1$$

$$\text{L}^1\text{-Norm} = \sum_{i=1}^{n} |w_i| = |w_1| + |w_2| + \cdots + |w_n|$$

代表模型的目標除了要讓 $\mathcal{L}$ 越小越好之外，
還要讓所有的 w 的絕對值和越小越好

L¹-Norm 說明：https://mathworld.wolfram.com/L1-Norm.html

Gradient Descent with L1 Regularization

$$\mathcal{L}_{\text{new}} = \mathcal{L} + \lambda \|\mathrm{w}\|_1$$

對任意 w_i 進行偏微分：

$$\frac{\partial \mathcal{L}_{\text{new}}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial w_i} + \lambda \cdot \text{sign}(w_i)$$

梯度下降：

$$w_i' = w_i - \eta \frac{\partial \mathcal{L}_{\text{new}}}{\partial w_i}$$

(代入) $$= w_i - \eta\left(\frac{\partial \mathcal{L}}{\partial w_i} + \lambda \cdot \text{sign}(w_i)\right)$$

每次更新都會扣減
(weight decay)

AdamW will be in Week 7

The paper of AdamW:

Published as a conference paper at ICLR 2019

DECOUPLED WEIGHT DECAY REGULARIZATION

Ilya Loshchilov & Frank Hutter
University of Freiburg
Freiburg, Germany,
{ilya,fh}@cs.uni-freiburg.de

Week7: 過擬合、正規化、模型訓練技巧、期末專案介紹

[Recap] Adam

- Adam: adaptive moment estimation
- 結合了 Momentum、Adagrad 和 RMSProp 的概念

Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." ICLR 2015. https://arxiv.org/abs/1412.6980

[Recap] 如何有效訓練深度學習模型？

Optimizer:

- 對梯度動手腳
  - Momentum
- 自動調整學習率
  - Adagrad
  - RMSprop
  - Adam

[Recap] Adam 流程

- Adam: Momentum、Adagrad 和 RMSProp 的概念

1. 計算梯度 $g_t \leftarrow \nabla_w f(w_{t-1})$
2. 以移動平均紀錄梯度 g_t (為了對梯度動手腳 -- Momentum)
3. 以移動平均紀錄梯度平方 g_t^2 (為了調整學習率 -- Adagrad)
4. 更新參數 $w_t \leftarrow w_{t-1} - \eta \hat{m}_t \frac{1}{\sqrt{\hat{v}_t + \epsilon}}$

Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." ICLR 2015. https://arxiv.org/abs/1412.6980

Adam

Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." ICLR 2015. https://arxiv.org/abs/1412.6980

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize Learning rate
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector) Momentum
$v_0 \leftarrow 0$ (Initialize 2nd moment vector) Adagrad, RMSProp
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters) Adagrad
end while
return θ_t (Resulting parameters)

[Adam]
m 記錄梯度
v 記錄梯度平方

移動平均取自 RMSProp

[實作細節] 把 L2 Regularization 加到 SGD

Loshchilov, Ilya, and Frank Hutter. “Decoupled Weight Decay Regularization.” International Conference on Learning Representations (2019).

1. 計算梯度 $g_t \leftarrow \nabla_w f(w_{t-1}) + \lambda w_{t-1}$ 證明請見 AdamW 論文 Proposition 1 proof
2. 更新參數 $w_t \leftarrow w_{t-1} - \eta g_t$

等價於

$$w_i' = w_i - \eta\left(\frac{\partial \mathcal{L}}{\partial w_i} + \lambda w_i\right)$$

[實作細節] 把 L2 Regularization 加到 Adam

在SGD正確，但在Adam是錯的，因為改變1的梯度會影響到梯度紀錄

1. 計算梯度 $g_t \leftarrow \nabla_w f(w_{t-1}) + \lambda w_{t-1}$
2. 以移動平均紀錄梯度 g_t (為了對梯度動手腳 -- Momentum)
3. 以移動平均紀錄梯度平方 g_t^2 (為了調整學習率 -- Adagrad)
4. 更新參數 $w_t \leftarrow w_{t-1} - \eta \hat{m}_t \frac{1}{\sqrt{\hat{v}_t} + \epsilon}$

Loshchilov, Ilya, and Frank Hutter. “Decoupled Weight Decay Regularization.” International Conference on Learning Representations (2019).

AdamW

分離的

AdamW: Adam with decoupled weight decay

1. 計算梯度 $g_t \leftarrow \nabla_w f(w_{t-1}) + \lambda w_{t-1}$
2. 以移動平均紀錄梯度 g_t (為了對梯度動手腳 -- Momentum)
3. 以移動平均紀錄梯度平方 g_t^2 (為了調整學習率 -- Adagrad)
4. 更新參數 $w_t \leftarrow w_{t-1} - \eta(\hat{m}_t \frac{1}{\sqrt{\hat{v}_t}+\epsilon} + \lambda w_{t-1})$

$$w_i' = w_i - \eta(\frac{\partial \mathcal{L}}{\partial w_i} + \lambda w_i)$$

Loshchilov, Ilya, and Frank Hutter. “Decoupled Weight Decay Regularization.” International Conference on Learning Representations (2019).

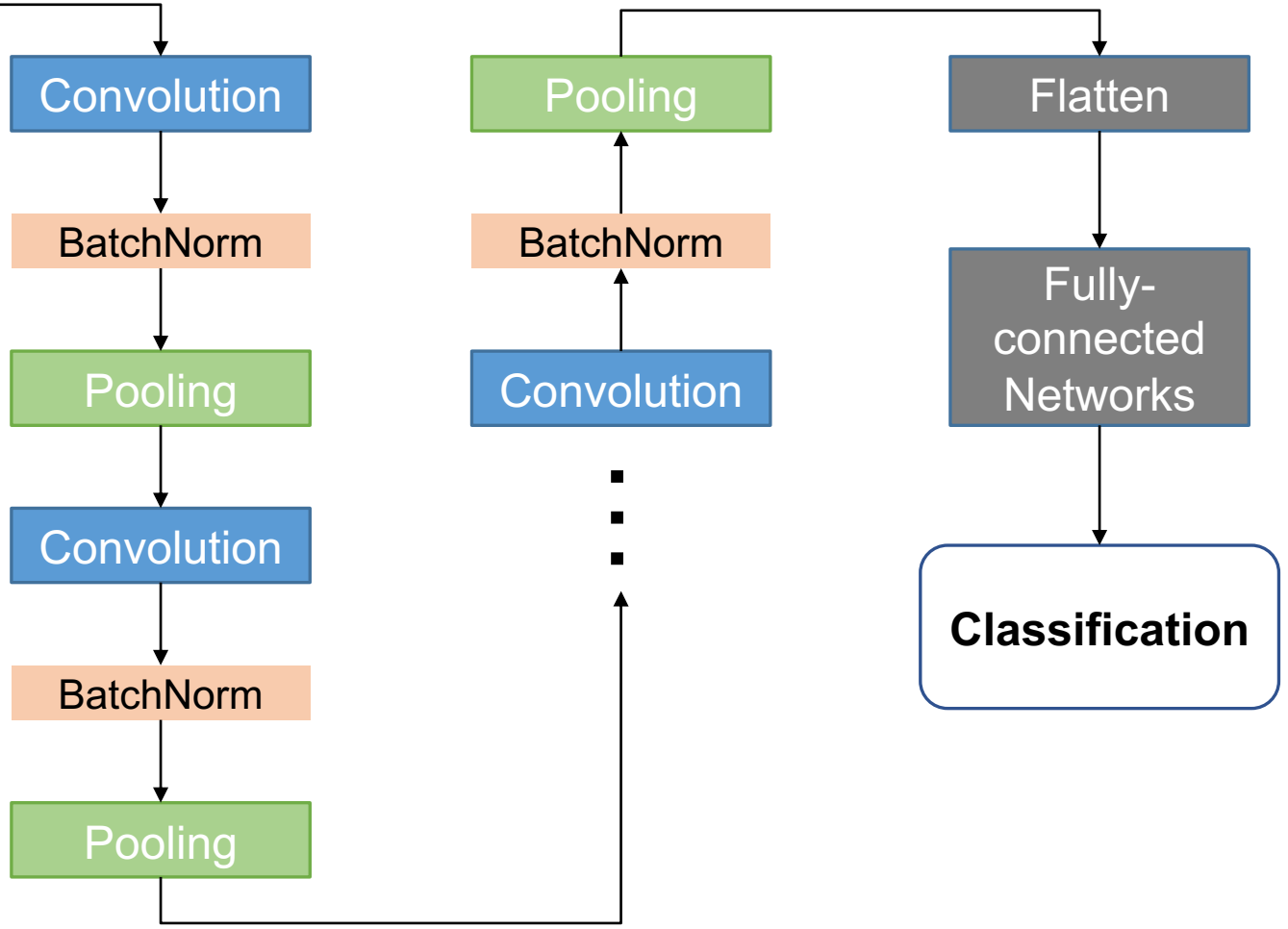
延伸學習

李宏毅老師影片

https://youtu.be/xki61j7z-30?si=ofWigNuHKkTdPC4f&t=3463

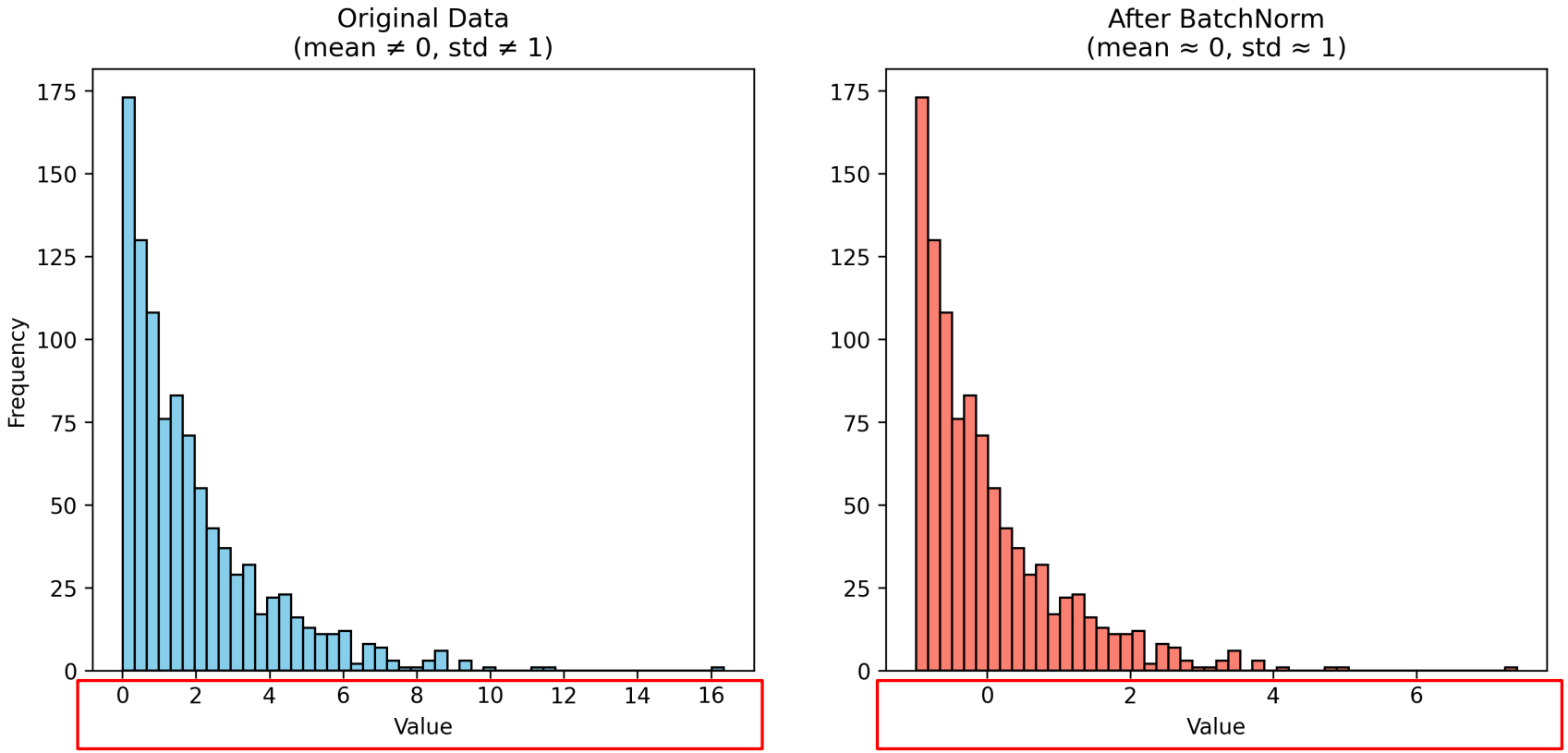
Batch Normalization

The whole CNN

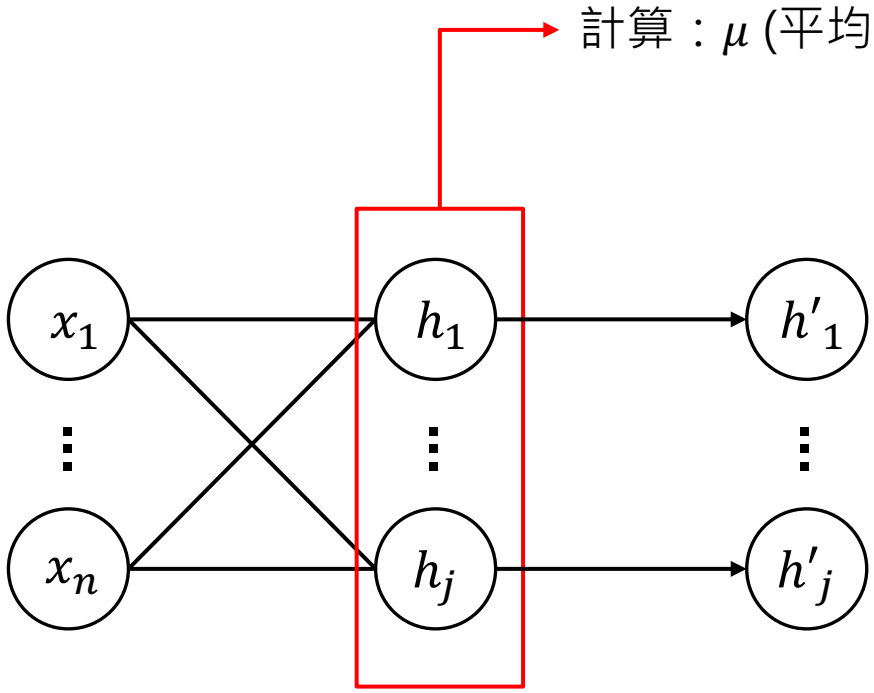


Batch Normalization (BatchNorm)

- BatchNorm 對數值進行轉換，使得轉換後的數值的平均值為零、標準差為1
- 類似於統計上的 Z-score standardization



MLP 中的 BatchNorm



$$h'_1 = \frac{h_1 - \mu}{\sigma}$$

$$h'_j = \frac{h_j - \mu}{\sigma}$$

Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." International conference on machine learning. 2015.

BatchNorm During Training

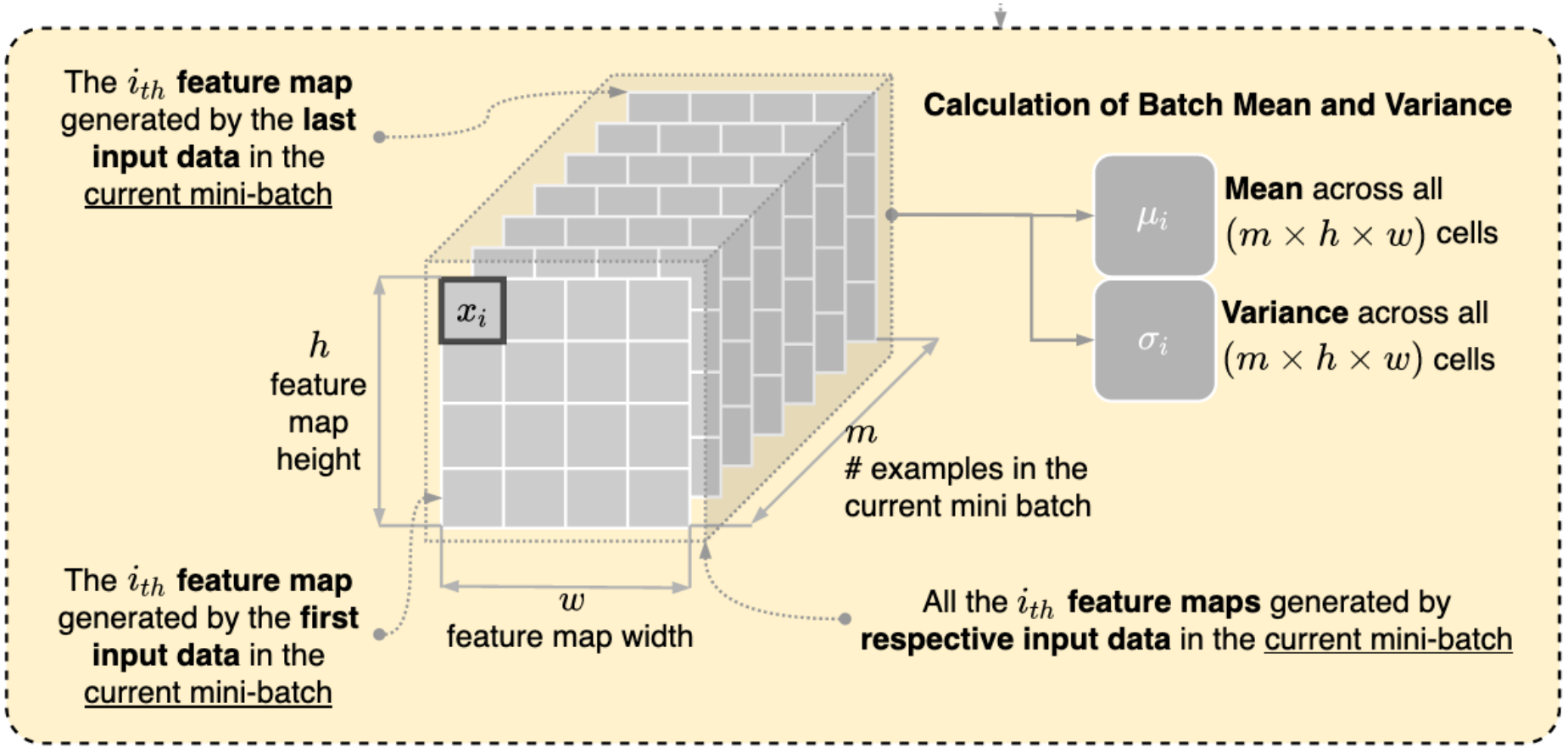
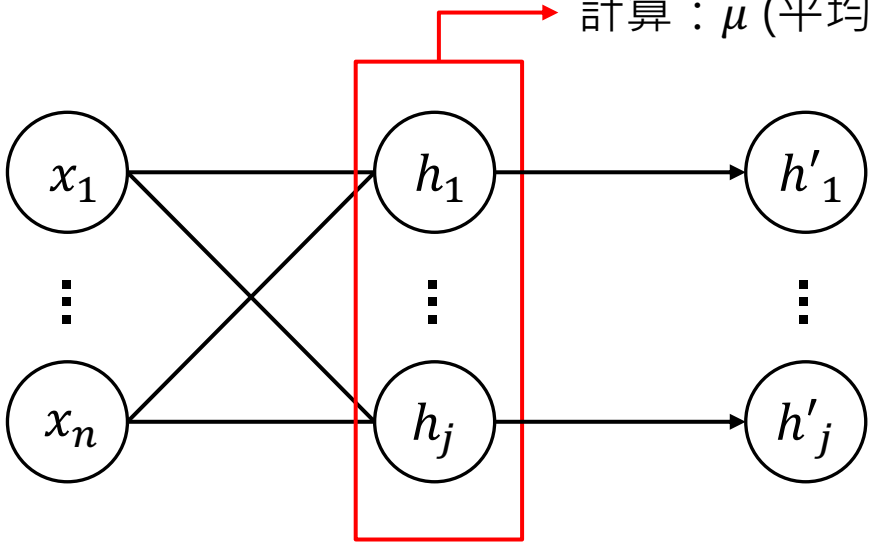


Figure source: https://stackoverow.com/questions/65613694/calculation-of-mean-and-variance-in-flbatch-normalization-in-convolutional-neural

BatchNorm During Test (Inference)

Training

計算：μ (平均數) & σ (標準差)



https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html

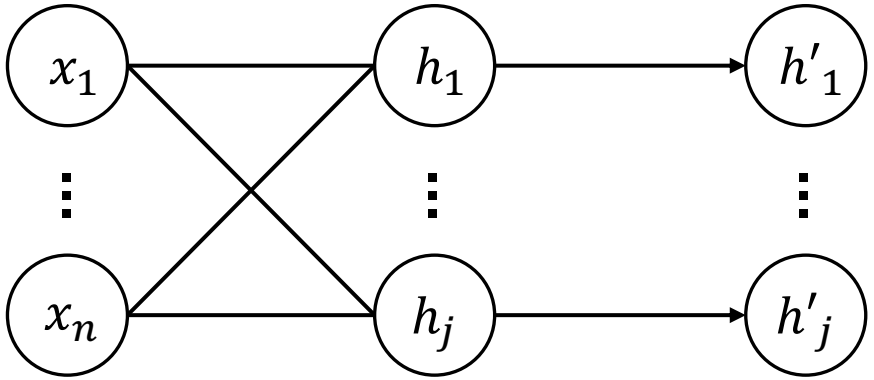
Moving average

$$\mu_t = \alpha\mu_t + (1-\alpha)\mu_{t-1}$$

$$\sigma_t = \alpha\sigma_t + (1-\alpha)\sigma_{t-1}$$

Inference

假設只有1張影像



$$h'_1 = \frac{h_1 - \mu_t}{\sigma_t}$$

$$h'_j = \frac{h_j - \mu_t}{\sigma_t}$$

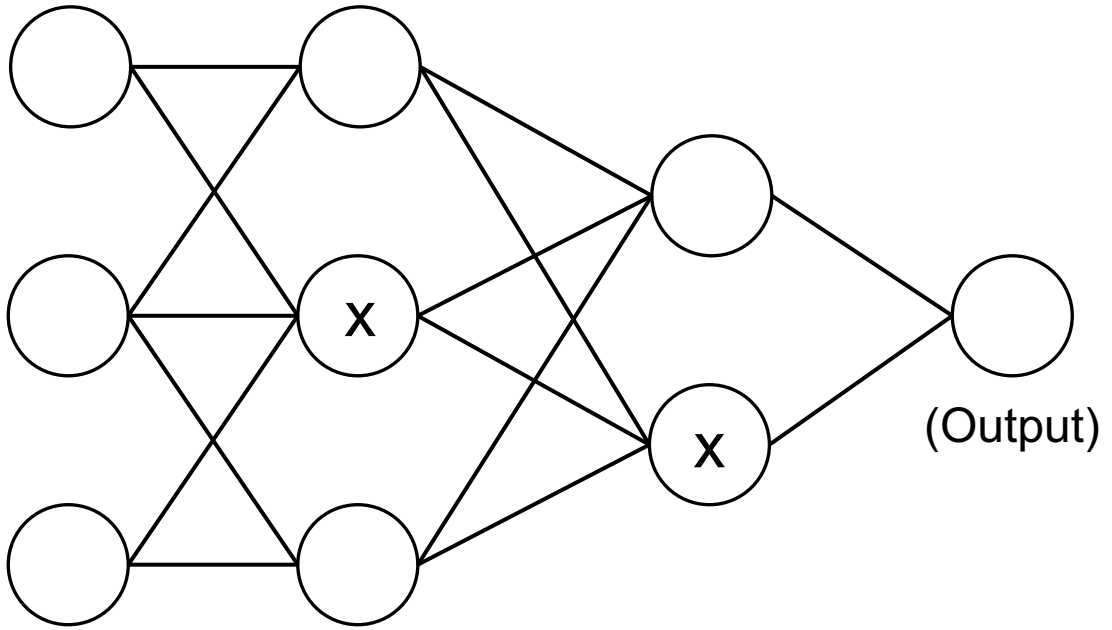
[觀察] BatchNorm

- BatchNorm 可以：
  - 減少不同特徵尺度帶來的偏差，避免梯度爆炸或消失
  - 提高模型訓練時的穩定性，加速收斂

Dropout

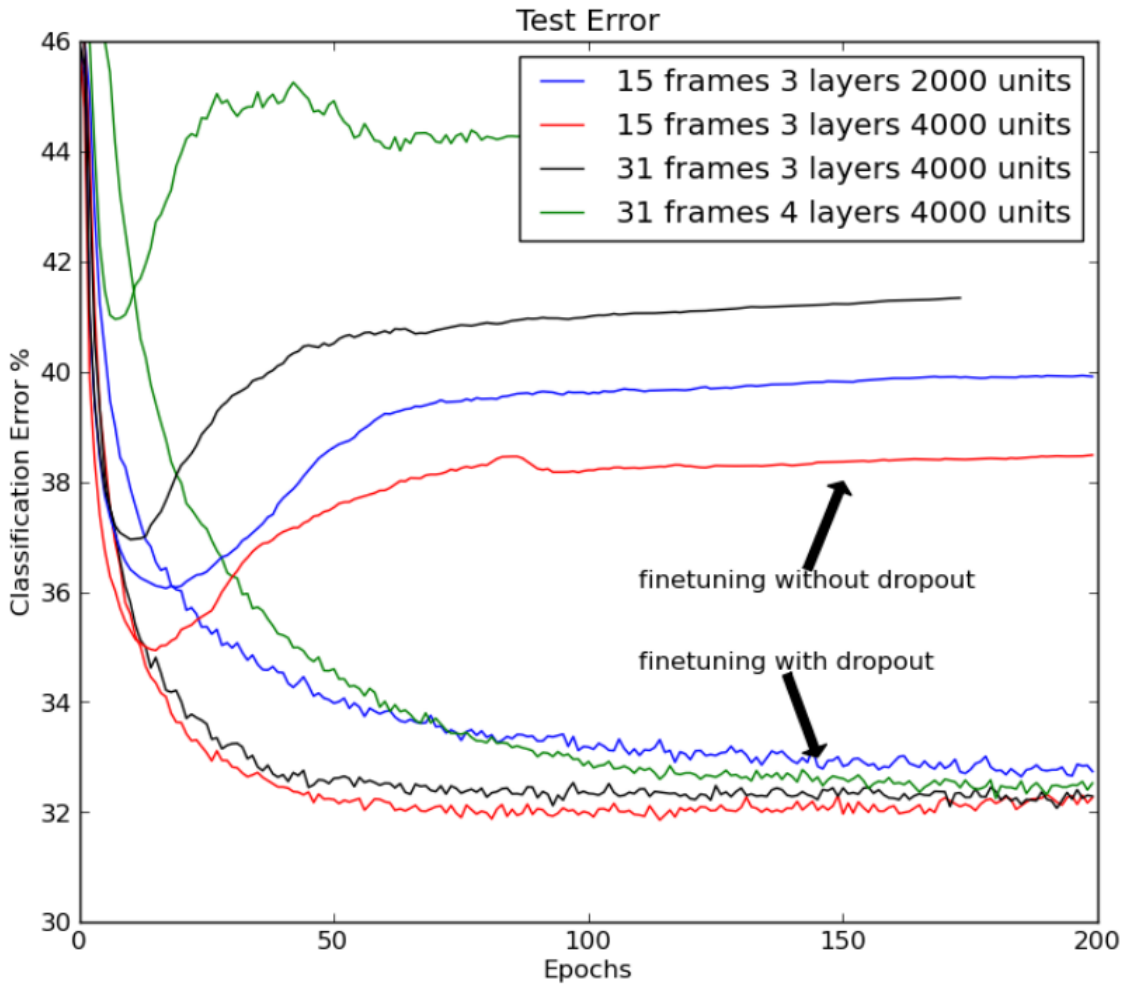
Dropout

- 訓練的時候把神經網路每一層的輸入值部份以 p 的機率隨機替換成 0
  - p 是超參數
- 可防止模型過擬合 (over-fitting)，增強模型的泛化能力 (generalization)



Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." The journal of machine learning research 15.1 (2014): 1929-1958.

Dropout may improve generalization

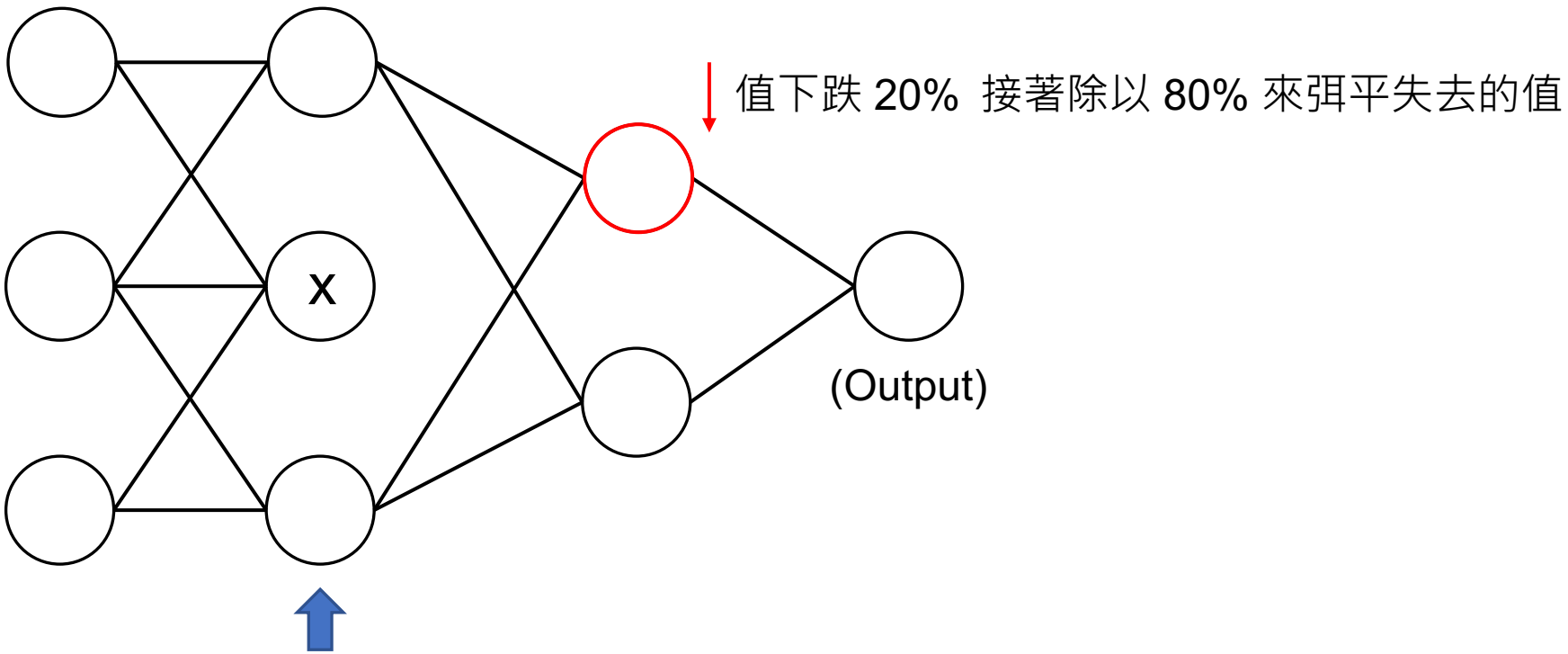


Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." The journal of machine learning research 15.1 (2014): 1929-1958.

Dropout Implementation

https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html

- 實務上採用 Inverted Dropout: output 要除以 (1 - p)
- 假設 p = 0.2:



model.eval()

- 使用 training 時取得的平均值跟標準差來算 batch normalization
- 關閉 dropout (保留所有的神經元)

Thank you!

Instructor: 林英嘉

✉ yjlin@cgu.edu.tw

TA: 劉美辰

✉ m1461014@cgu.edu.tw